UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

CS2023 Algoritmos y Estructuras de Datos
Implementación de Listas
Semana 3

Víctor Racsó Galván Oyola

vgalvan@utec.edu.pe

25 de agosto de 2026

¿Qué aprenderemos hoy?

¿Qué aprenderemos hoy?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- El nodo y el puntero en C++

¿Qué aprenderemos hoy?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- El nodo y el puntero en C++
- Implementando la Lista Simple

¿Qué aprenderemos hoy?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- El nodo y el puntero en C++
- Implementando la Lista Simple
- Implementando la Lista Doble

¿Qué aprenderemos hoy?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- El nodo y el puntero en C++
- Implementando la Lista Simple
- Implementando la Lista Doble
- Implementando la Lista Circular

¿Qué aprenderemos hoy?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- El nodo y el puntero en C++
- Implementando la Lista Simple
- Implementando la Lista Doble
- Implementando la Lista Circular
- Extra: listas **sin** punteros, sobre un arreglo

El nodo y el puntero

El nodo: una `struct` con un puntero a sí misma

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

```
template <typename T>
struct Nodo {
    T dato;
    Nodo* siguiente;
    Nodo(T d) : dato(d), siguiente(nullptr) {}
};
```

El nodo: una `struct` con un puntero a sí misma

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

```
template <typename T>
struct Nodo {
    T dato;
    Nodo* siguiente;
    Nodo(T d) : dato(d), siguiente(nullptr) {}
};
```

Memoria dinámica

Cada nodo se crea con `new` (vive en el *heap*, no en la pila) y se destruye con `delete`. A diferencia de un `vector`, **nosotros** somos responsables de administrar esa memoria.

Creando y enlazando nodos a mano

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

```
Nodo<int>* a = new Nodo<int>(5);
Nodo<int>* b = new Nodo<int>(3);
a->siguiente = b;       // enlazamos a -> b
b->siguiente = nullptr; // b es el ultimo
```

Creando y enlazando nodos a mano

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

```
Nodo<int>* a = new Nodo<int>(5);
Nodo<int>* b = new Nodo<int>(3);
a->siguiente = b;       // enlazamos a -> b
b->siguiente = nullptr; // b es el ultimo
```

¡Piénsalo!

Si pierdes la única variable que apunta a un nodo creado con `new` (por ejemplo, la sobrescribes sin antes hacer `delete`), ¿qué pasa con esa memoria?

Creando y enlazando nodos a mano

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

```
Nodo<int>* a = new Nodo<int>(5);
Nodo<int>* b = new Nodo<int>(3);
a->siguiente = b;       // enlazamos a -> b
b->siguiente = nullptr; // b es el ultimo
```

¡Piénsalo!

Si pierdes la única variable que apunta a un nodo creado con `new` (por ejemplo, la sobrescribes sin antes hacer `delete`), ¿qué pasa con esa memoria?

Se queda reservada para siempre, inalcanzable: es una **fuga de memoria** (*memory leak*).

Lista Simple

Estructura de la lista

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

```
template <typename T>
struct ListaSimple {
    Nodo<T>* head = nullptr;
    Nodo<T>* tail = nullptr;
    int n = 0;

    bool empty() const { return n == 0; }
    int size() const   { return n; }
```

Estructura de la lista

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

```
template <typename T>
struct ListaSimple {
    Nodo<T>* head = nullptr;
    Nodo<T>* tail = nullptr;
    int n = 0;

    bool empty() const { return n == 0; }
    int size() const   { return n; }
```

Guardamos `tail` (puntero al último nodo) para que `push_back` también sea $O(1)$ — sin él, habría que recorrer toda la lista para encontrar el final.

Lista Simple: pseudocódigo de `push`

Algoritmo 1: PUSH_FRONT(L, x)

nuevo $\leftarrow$ nodo con dato x, nuevo.siguiente $\leftarrow$ $L.head$;
$L.head \leftarrow$ nuevo ;
si $L.tail = nulo$ **entonces**
  $L.tail \leftarrow$ nuevo

Lista Simple: pseudocódigo de `push`

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Algoritmo 3: PUSH_FRONT(L, x)

nuevo $\leftarrow$ nodo con dato x, nuevo.siguiente $\leftarrow$ $L.head$;
$L.head \leftarrow$ nuevo ;
si $L.tail = nulo$ **entonces**
  $L.tail \leftarrow$ nuevo

Algoritmo 4: PUSH_BACK(L, x)

nuevo $\leftarrow$ nodo con dato x, nuevo.siguiente $\leftarrow$ nulo ;
si $L.tail = nulo$ **entonces**
  $L.head \leftarrow L.tail \leftarrow$ nuevo
en otro caso
  $L.tail$.siguiente $\leftarrow$ nuevo ; $L.tail \leftarrow$ nuevo

push_front y push_back

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

```
void push_front(T x) {
    Nodo<T>* nuevo = new Nodo<T>(x);
    nuevo->siguiente = head;
    head = nuevo;
    if (tail == nullptr) tail = nuevo;
    n++;
}

void push_back(T x) {
    Nodo<T>* nuevo = new Nodo<T>(x);
    if (tail == nullptr) head = tail = nuevo;
    else { tail->siguiente = nuevo; tail = nuevo; }
    n++;
}
```

Lista Simple: pseudocódigo de Insertar_Después

Algoritmo 5: INSERTAR_DESPUÉS(p, x)

nuevo $\leftarrow$ nodo con dato x ;
nuevo.siguiente $\leftarrow$ p.siguiente ;
p.siguiente $\leftarrow$ nuevo ;

Insertar después de un nodo dado

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

```
// inserta x justo despues del nodo p, en O(1)
void insertar_despues(Nodo<T>* p, T x) {
    Nodo<T>* nuevo = new Nodo<T>(x);
    nuevo->siguiente = p->siguiente;
    p->siguiente = nuevo;
    if (p == tail) tail = nuevo;
    n++;
}
```

Insertar después de un nodo dado

```
// inserta x justo despues del nodo p, en O(1)
void insertar_despues(Nodo<T>* p, T x) {
    Nodo<T>* nuevo = new Nodo<T>(x);
    nuevo->siguiente = p->siguiente;
    p->siguiente = nuevo;
    if (p == tail) tail = nuevo;
    n++;
}
```

Es $O(1)$ porque **ya tenemos** el puntero p — no incluye el costo de **encontrarlo**, que sí sería $O(n)$ si solo tuviéramos su posición.

Lista Simple: pseudocódigo de Pop_Front

Algoritmo 6: POP_FRONT(L)

viejo $\leftarrow$ $L.head$;
$L.head \leftarrow L.head$.siguiente ;
si $L.head = nulo$ **entonces**
  $L.tail \leftarrow$ nulo
liberar viejo ;

Eliminar el primer nodo




```
void pop_front() {
    Nodo<T>* viejo = head;
    head = head->siguiente;
    if (head == nullptr) tail = nullptr;  // quedo vacia
    delete viejo;
    n--;
}
```

Eliminar el primer nodo




```
void pop_front() {
    Nodo<T>* viejo = head;
    head = head->siguiente;
    if (head == nullptr) tail = nullptr;  // quedo vacia
    delete viejo;
    n--;
}
```

¡Piénsalo!

Para eliminar el nodo **siguiente** a uno dado `p` (no el primero), ¿qué puntero necesitas guardar **antes** de hacer `delete`?

Eliminar el primer nodo




```
void pop_front() {
    Nodo<T>* viejo = head;
    head = head->siguiente;
    if (head == nullptr) tail = nullptr;  // quedo vacia
    delete viejo;
    n--;
}
```

¡Piénsalo!

Para eliminar el nodo **siguiente** a uno dado `p` (no el primero), ¿qué puntero necesitas guardar **antes** de hacer `delete`?

`p->siguiente->siguiente` — hay que reconectar `p` con lo que queda **después** del nodo eliminado, antes de liberarlo.

Lista Simple: pseudocódigo de Buscar

Algoritmo 7: BUSCAR(L, x)

```
p ← L.head ;
mientras p ≠ nulo y p.dato ≠ x hacer
    p ← p.siguiente ;
devolver p ;                                  // nulo si no se encontró
```

Recorrer y buscar

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

```
Nodo<T>* buscar(T x) const {
    Nodo<T>* p = head;
    while (p != nullptr && p->dato != x) p = p->siguiente;
    return p;  // nullptr si no se encontro
}

void imprimir() const {
    for (Nodo<T>* p = head; p != nullptr; p = p->siguiente)
        cout << p->dato << " ";
    cout << "\n";
}
```

El destructor: liberar todo

```
    ~ListaSimple() {
        Nodo<T>* p = head;
        while (p != nullptr) {
            Nodo<T>* siguiente = p->siguiente;
            delete p;
            p = siguiente;
        }
    }
};
```

El destructor: liberar todo

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

```
    ~ListaSimple() {
        Nodo<T>* p = head;
        while (p != nullptr) {
            Nodo<T>* siguiente = p->siguiente;
            delete p;
            p = siguiente;
        }
    }
};
```

¡Piénsalo!

¿Por qué hay que guardar `p->siguiente` **antes** de hacer `delete p`?

El destructor: liberar todo

```
    ~ListaSimple() {
        Nodo<T>* p = head;
        while (p != nullptr) {
            Nodo<T>* siguiente = p->siguiente;
            delete p;
            p = siguiente;
        }
    }
};
```

¡Piénsalo!

¿Por qué hay que guardar `p->siguiente` **antes** de hacer `delete p`?

Porque `delete p` libera esa memoria: leer `p->siguiente` **después** sería acceder a memoria ya liberada (comportamiento indefinido).

Lista Doble

El nodo doble

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

```
template <typename T>
struct NodoDoble {
    T dato;
    NodoDoble* siguiente;
    NodoDoble* anterior;
    NodoDoble(T d) : dato(d),
                     siguiente(nullptr),
                     anterior(nullptr) {}
};
```

El nodo doble

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

```
template <typename T>
struct NodoDoble {
    T dato;
    NodoDoble* siguiente;
    NodoDoble* anterior;
    NodoDoble(T d) : dato(d),
                     siguiente(nullptr),
                     anterior(nullptr) {}
};
```

Un puntero más que la lista simple — ese único cambio es lo que permite eliminar en $O(1)$ dado solo el nodo.

Lista Doble: pseudocódigo de `push_back`

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Algoritmo 8: PUSH_BACK(L, x) — doble

nuevo $\leftarrow$ nodo con dato x, anterior $\leftarrow$ $L.tail$;
si $L.tail \neq nulo$ **entonces**
  $L.tail$.siguiente $\leftarrow$ nuevo
en otro caso
  $L.head \leftarrow$ nuevo
$L.tail \leftarrow$ nuevo ;

`push_back` en la lista doble

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

```
void push_back(T x) {
    NodoDoble<T>* nuevo = new NodoDoble<T>(x);
    nuevo->anterior = tail;
    if (tail != nullptr) tail->siguiente = nuevo;
    else head = nuevo;
    tail = nuevo;
    n++;
}
```

`push_back` en la lista doble

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

```
void push_back(T x) {
    NodoDoble<T>* nuevo = new NodoDoble<T>(x);
    nuevo->anterior = tail;
    if (tail != nullptr) tail->siguiente = nuevo;
    else head = nuevo;
    tail = nuevo;
    n++;
}
```

Igual que antes, pero ahora también hay que dejar `anterior` apuntando correctamente — un puntero más que mantener consistente en cada operación.

Lista Doble: pseudocódigo de Eliminar

Algoritmo 9: ELIMINAR(p) — doble, $O(1)$

si $p.anterior \neq nulo$ **entonces**
  p.anterior.siguiente $\leftarrow$ p.siguiente
en otro caso
  $L.head \leftarrow p$.siguiente
si $p.siguiente \neq nulo$ **entonces**
  p.siguiente.anterior $\leftarrow$ p.anterior
en otro caso
  $L.tail \leftarrow p$.anterior
liberar p ;

Eliminar dado el propio puntero al nodo: $O(1)$




```
void eliminar(NodoDoble<T>* p) {
    if (p->anterior) {
        p->anterior->siguiente = p->siguiente;
    }
    else head = p->siguiente;

    if (p->siguiente) {
        p->siguiente->anterior = p->anterior;
    }
    else tail = p->anterior;

    delete p;
    n--;
}
```

Eliminar dado el propio puntero al nodo: $O(1)$

```
void eliminar(NodoDoble<T>* p) {
    if (p->anterior) {
        p->anterior->siguiente = p->siguiente;
    }
    else head = p->siguiente;

    if (p->siguiente) {
        p->siguiente->anterior = p->anterior;
    }
    else tail = p->anterior;

    delete p;
    n--;
}
```

Esto era exactamente lo que **no** podíamos hacer en $O(1)$ en la lista simple: aquí `p->anterior` ya está guardado, no hace falta buscarlo.

Lista Circular

Lista Circular: pseudocódigo de `push_back`

Algoritmo 10: PUSH_BACK(L, x) — circular

```
nuevo ← nodo con dato x ;
si L.head = nulo entonces
    L.head ← nuevo ;
    nuevo.siguiente ← nuevo ;                    // se apunta a sí mismo
en otro caso
    nuevo.siguiente ← L.head ;
    L.tail.siguiente ← nuevo ;
L.tail ← nuevo ;
```

Circular simple: el último apunta al primero




```
void push_back(T x) {
    Nodo<T>* nuevo = new Nodo<T>(x);
    if (head == nullptr) {
        head = nuevo;
        nuevo->siguiente = nuevo;  // se apunta a si mismo
    } else {
        nuevo->siguiente = head;
        tail->siguiente = nuevo;
    }
    tail = nuevo;
    n++;
}
```

Lista Circular: pseudocódigo de Recorrer

Algoritmo 11: RECORRER(L) — circular

```
si L.head = nulo entonces
    devolver
p ← L.head ;
repetir
    visitar p.dato ;
    p ← p.siguiente ;
hasta que p = L.head;
```

Lista Circular: pseudocódigo de Recorrer

Algoritmo 12: RECORRER(L) — circular

```
si L.head = nulo entonces
    devolver
p ← L.head ;
repetir
    visitar p.dato ;
    p ← p.siguiente ;
hasta que p = L.head;
```

REPEAT...UNTIL en pseudocódigo es exactamente `do-while` en C++: ejecuta el cuerpo **primero**, y recién después evalúa la condición de parada.

Cuidado al recorrer: no hay `nullptr`

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

```
void imprimir() const {
    if (head == nullptr) return;
    Nodo<T>* p = head;
    do {
        cout << p->dato << " ";
        p = p->siguiente;
    } while (p != head);   // NO: while (p != nullptr)
}
```

Cuidado al recorrer: no hay `nullptr`




```
void imprimir() const {
    if (head == nullptr) return;
    Nodo<T>* p = head;
    do {
        cout << p->dato << " ";
        p = p->siguiente;
    } while (p != head);    // NO: while (p != nullptr)
}
```

Usamos `do-while` (no `while`) porque `p` **empieza** siendo `head` — si usáramos `while(p != head)` desde el inicio, el ciclo nunca se ejecutaría.

Extra: listas sin punteros

¿Y si no queremos usar `new/delete`?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

La idea

En vez de un puntero real, guardamos el "siguiente" como un **índice entero** dentro de un arreglo paralelo. El arreglo entero actúa como la memoria dinámica: cada posición es un nodo, y "-1" representa $\emptyset$.

¿Y si no queremos usar `new/delete`?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

La idea

En vez de un puntero real, guardamos el "siguiente" como un **índice entero** dentro de un arreglo paralelo. El arreglo entero actúa como la memoria dinámica: cada posición es un nodo, y "-1" representa $\emptyset$.

¿Por qué haría esto alguien?

Sin `new/delete` en cada operación (más rápido en la práctica), mejor localidad de memoria (los nodos viven en un solo arreglo contiguo), y es un patrón muy usado en programación competitiva con memoria estática.

Pseudocódigo: lista enlazada sobre arreglos

Algoritmo 13: NUEVO_NODO(x)

$idx \leftarrow$ libre ;
libre $\leftarrow$ libre $+1$;
dato[idx] $\leftarrow x$;
siguiente[idx] $\leftarrow -1$;
devolver idx ;

Pseudocódigo: lista enlazada sobre arreglos

Algoritmo 15: NUEVO_NODO(x)

$idx \leftarrow$ libre ;
libre $\leftarrow$ libre $+1$;
dato[idx] $\leftarrow x$;
siguiente[idx] $\leftarrow -1$;
devolver idx ;

Algoritmo 16: PUSH_FRONT(x) — sobre arreglos

$idx \leftarrow$ NUEVO_NODO(x) ;
siguiente[idx] $\leftarrow head$;
$head \leftarrow idx$;

Lista enlazada simulada sobre arreglos

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

```
int dato[MAXN], siguiente[MAXN];
int head = -1, libre = 0;

int nuevo_nodo(int x) {
    int idx = libre++;
    dato[idx] = x;
    siguiente[idx] = -1;
    return idx;
}

void push_front(int x) {
    int idx = nuevo_nodo(x);
    siguiente[idx] = head;
    head = idx;
}
```

¡Piénsalo!

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Reciclando posiciones eliminadas

Si eliminas el nodo en la posición `i`, esa posición del arreglo queda "libre" pero `libre` ya avanzó de largo. ¿Cómo reutilizarías esa posición para el siguiente `nuevo_nodo`, sin desperdiciarla?

¡Piénsalo!

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Reciclando posiciones eliminadas

Si eliminas el nodo en la posición `i`, esa posición del arreglo queda "libre" pero `libre` ya avanzó de largo. ¿Cómo reutilizarías esa posición para el siguiente `nuevo_nodo`, sin desperdiciarla?

La idea: una *lista libre*

Mantener **otra** lista enlazada (¡sobre el mismo arreglo `siguiente`!) que conecta las posiciones eliminadas entre sí. `nuevo_nodo` primero revisa esa lista antes de usar `libre++`.

Punteros dinámicos vs. índices en un arreglo

	Punteros (`new/delete`)	**Índices en arreglo**

Punteros dinámicos vs. índices en un arreglo

	Punteros (`new/delete`)	**Índices en arreglo**
Memoria	dinámica, dispersa	estática, contigua

Punteros dinámicos vs. índices en un arreglo

	Punteros (`new/delete`)	**Índices en arreglo**
Memoria	dinámica, dispersa	estática, contigua
Localidad de caché	mala	buena

Punteros dinámicos vs. índices en un arreglo

	Punteros (`new/delete`)	**Índices en arreglo**
Memoria	dinámica, dispersa	estática, contigua
Localidad de caché	mala	buena
Reciclar espacio liberado	automático (`delete`)	manual (lista libre)

Punteros dinámicos vs. índices en un arreglo

	Punteros (`new/delete`)	**Índices en arreglo**
Memoria	dinámica, dispersa	estática, contigua
Localidad de caché	mala	buena
Reciclar espacio liberado	automático (`delete`)	manual (lista libre)
Tamaño máximo	limitado por RAM disponible	fijo, hay que reservarlo antes

Punteros dinámicos vs. índices en un arreglo

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

	Punteros (`new/delete`)	**Índices en arreglo**
Memoria	dinámica, dispersa	estática, contigua
Localidad de caché	mala	buena
Reciclar espacio liberado	automático (`delete`)	manual (lista libre)
Tamaño máximo	limitado por RAM disponible	fijo, hay que reservarlo antes

Misma idea de fondo (nodos + "siguiente"), dos formas muy distintas de administrar la memoria detrás.

Resumen de la sesión



Resumen de la sesión

- Un nodo es una `struct` con su dato y un puntero `siguiente` (y `anterior`, en la doble), creado con `new` y liberado con `delete`.

Resumen de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Un nodo es una `struct` con su dato y un puntero `siguiente` (y `anterior`, en la doble), creado con `new` y liberado con `delete`.
- Guardar `head` y `tail` permite que insertar en ambos extremos sea $O(1)$.

Resumen de la sesión

- Un nodo es una `struct` con su dato y un puntero `siguiente` (y `anterior`, en la doble), creado con `new` y liberado con `delete`.
- Guardar `head` y `tail` permite que insertar en ambos extremos sea $O(1)$.
- La lista doble hace $O(1)$ lo que en la simple era $O(n)$: eliminar dado solo el puntero al nodo.

Resumen de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Un nodo es una `struct` con su dato y un puntero `siguiente` (y `anterior`, en la doble), creado con `new` y liberado con `delete`.
- Guardar `head` y `tail` permite que insertar en ambos extremos sea $O(1)$.
- La lista doble hace $O(1)$ lo que en la simple era $O(n)$: eliminar dado solo el puntero al nodo.
- La circular se recorre con `do-while`, comparando contra `head` en vez de contra `nullptr`.

Resumen de la sesión

- Un nodo es una `struct` con su dato y un puntero `siguiente` (y `anterior`, en la doble), creado con `new` y liberado con `delete`.
- Guardar `head` y `tail` permite que insertar en ambos extremos sea $O(1)$.
- La lista doble hace $O(1)$ lo que en la simple era $O(n)$: eliminar dado solo el puntero al nodo.
- La circular se recorre con `do-while`, comparando contra `head` en vez de contra `nullptr`.
- Las listas enlazadas también se pueden simular **sin punteros**, usando índices dentro de un arreglo — la misma idea, con memoria estática y mejor localidad.

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Gracias